

URP 연구활동 최종 발표

URP Final Presentation.

Vortex GPGPU warp scheduler별 워크로드 병목 분석, 최적화

배경 & 목표

연구 목표

오픈소스 RISC-V GPGPU Vortex의 cycle 단위 C++ 시뮬레이터(SimX)에서 워프 스케줄러 정책을 **구현·분석**하고, **워크로드 특성에 따라 가장 적합한 정책을 도출**한다.

1

스케줄러 정책 구현·분석

RR · GTO · gCAWS · iPAWS를 SimX에 구현하고 비교

2

병목 분석 / 완화 → IPC 향상

정책 변경으로 하드웨어 자원 병목을 줄여 전체 IPC 개선

3

최적화

시뮬레이터 검증을 실제 하드웨어 설계 (Verilog RTL)까지 연결

Configuration 선정

상용 GPU(RTX 3090) 1/20 downscale → Vortex에 맞는 구성으로 전환

1/20 Downscale (RTX 3090)	값
Cluster / Core / Lane / Warp	1 / 16 / 16 / 32
L1 D-cache	128 KB · 128B line · 16 MSHR · WT · 8-way
L3 cache	256 KB · write-back · 16-way

문제

- compute-bound sgemm3조차 L1 miss로 MSHR 마비
- 시뮬레이션 시간이 과도하게 길어짐
- memory·compute 워크로드 모두 메모리 병목

통찰

- 상용 GPU 구조를 그대로 따르는 것은 부적절 → Vortex에 맞는 구성 필요
- 이후 분석은 스케줄러 간 차이를 잘 드러내는 구성 선정에 초점

관련 연구 (Related Work)

CAWA / gCAWS

Lee et al., ISCA 2015

Coordinated Warp Scheduling & Cache Prioritization

$CPL(nInst \cdot CPI \cdot nStall)$ 로 critical warp 예측
→ gCAWS + CACP. kmeans 약 3배 향상

iPAWS

Lee et al., HPCA 2016

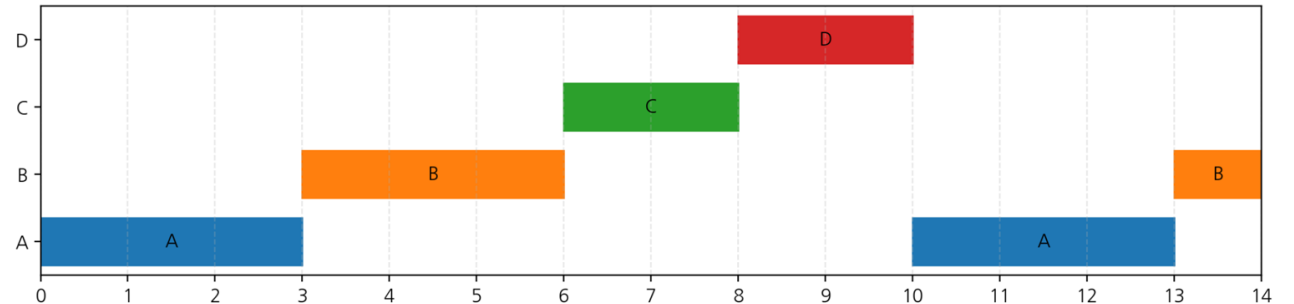
Issue-Pattern based Adaptive Scheduling

issue 패턴 concave→GTO / convex→RR,
WOI 식별. CCWS 단독 대비 평균 ~40% 향상

스케줄러 구현 & 제안

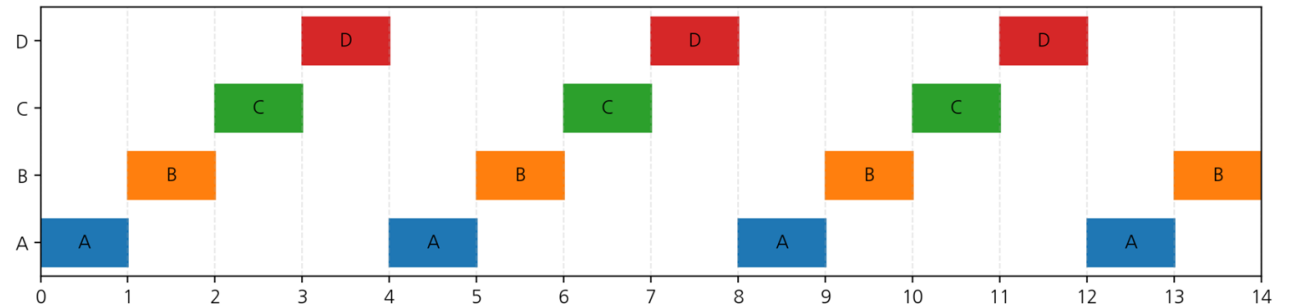
GTO (Greedy-than-oldest)

Stall이 걸릴 때까지 실행할 워프를 Greedy하게 실행하다
Stall이 걸리면 가장 오래된 (먼저 생성된) 워프 먼저 실행



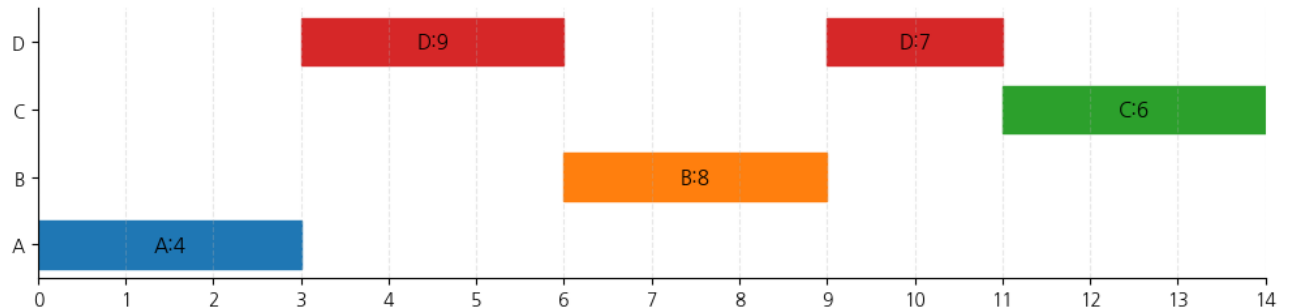
RR (Round-Robin)

ready 워프를 wid 순서로 번갈아(round-robin) 선택,
연속 접근이 여러 워프로 분산되어 inter-warp locality 활용은
약함



gCAWS (greedy Criticality-Aware Warp Schedule)

Greedy하게 실행 후 stall이 걸리면
늦게 끝날 것으로 예상되는 (criticality score가 높은)
워프를 먼저 실행



스케줄러 구현 & 제안

스케줄러	핵심 동작	비고
RR	Loosely Round-Robin — 다음 wid 선호	baseline
GTO	Greedy-Then-Oldest — 현재 wid 유지	locality
gCAWS	criticality = nInst·CPI + nStall 최댓값, tie=oldest, greedy	CAWA 기반
iPAWS	Adapt(GTO 측정) → Decide(convex/concave) → Execute	fusion

제안 iPAWS의 Execute phase에서 GTO 자리에 **gCAWS**를 적용 → 최종적으로 gCAWS·RR 중 워크로드에 맞는 정책을 동적 선택. GTO 단독보다 큰 향상 + iPAWS robustness 유지.

한계: 원 논문은 GPGPU-Sim 기반, 본 연구는 Vortex(RISC-V ISA · toolchain) 기반 → 기법을 완전히 동일하게 재현하기는 어려움.

스케줄러 구현 & 제안

구현한 baseline(RR·GTO)과 논문 기법 (gCAWS·iPAWS)

구현 완료 — 정상 동작 검증 (arbiter)

RR	Loosely Round-Robin · 다음 wid 선호	baseline
GTO	Greedy-Then-Oldest · 현재 wid 유지	locality 효과

논문 기법 재현 시도 — 여러 제약으로 완전한 재현 불가

gCAWS	criticality = nInst·CPI + nStall, tie=oldest (CAWA)	nInst 근사
iPAWS	issue 패턴(convex/concave) 적응, Execute=gCAWS	대부분 RR 선택

의도한 설계: iPAWS의 Execute phase에서 GTO 자리에 gCAWS 적용 → gCAWS·RR 중 동적 선택.

gCAWS 구현 한계 — CAWA nInst의 ISA 종속성

PTX의 @p0 bra 한 명령이 RISC-V에선 vx_split · beqz · vx_join 3개로 쪼개진다

RISC-V / Vortex disasm — kmeans (km_base.dis)

```
000000bc <kmeans_kernel>:
    bc:  vx_split_n a7, t0      ① divergence 시작
    c4:  beqz      t0, 0xc4     ② scalar branch · byte
    ——— .LBB0_6 loop: flw·flw·fsub·fmadd ———
100:  beqz      t4, 0x100     ③ loop branch · byte
104:  vx_join    a7
114:  vx_split   t0, t1       ⑤ 또 다른 divergence
118:  bnez      t1, 0x118     ⑥ branch
```

PTX / GPGPU-Sim

```
@p0 bra $L2
```

\$L2 = 명령어 인덱스

→ (target - branch) = path 명령어 수

0xc4 = byte 주소

→ 명령어 수 아님 (÷4 근사 · 압축이면 깨짐)

결과 CAWA의 nInst(branch path 명령어 수)를 정확하게 측정 불가.

Criticality = nInst·CPI + nStall . → CAWA 알고리즘이 GPGPU-Sim PTX ISA에 종속적.

- nInst = nextPC - currPC

원인 배제 & contention 재현

배제한 후보 (순차 접근에서 효과 없음 / 역효과)

- DDR3 latency 변경 — 효과 없음
- DRAM bank 수 변경 — 순차 접근 → 효과 없음
- L2 비활성화 — 효과 없음

MSHR = 2로 contention 재현 (NUM_WARPS=8)

gCAWS vs RR

+8.18%

kmeans (f96) — IPC 7.78 → 8.42

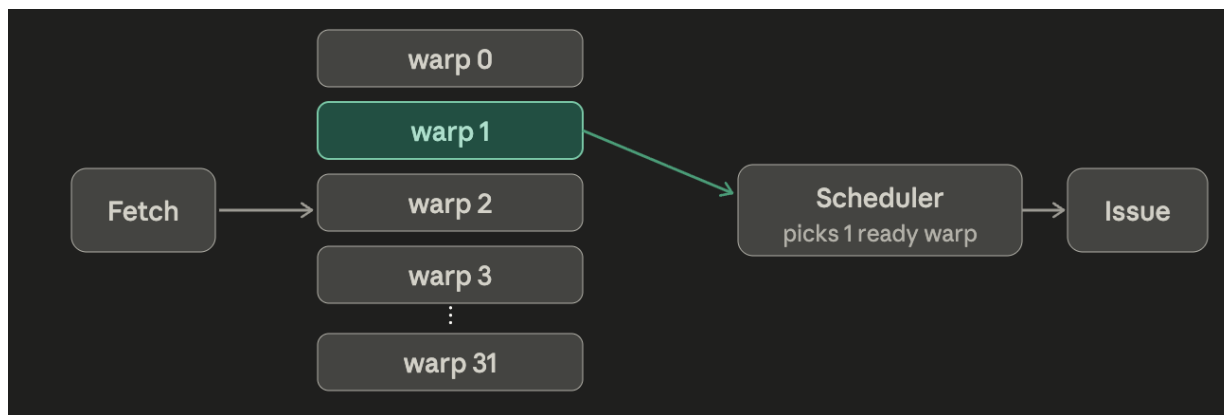
+9.24%

BFS (hh16k) — IPC 2.45 → 2.67

CAWA의 GDDR5 contention을 인위적으로 재현.

Scheduler insensitivity

기본 환경에서 RR·GTO·gCAWS IPC 차이는 noise 수준 (전구간 0.39% · region 2.15%)



scoreboard_block_ratio ≈ 97%

(kmeans) 후보 warp 대부분이 load 응답 대기

ready ≤ 1 cycle ≈ 90% · ready ≥ 2 = 9%

스케줄러가 선택을 수행할 순간 자체가 거의 없음

ready warp 수	RR	GTO
0	77%	78%
1	13%	14%
2 – 3	8%	7%
≥ 2 (스케줄 가능)	9%	9%

* 구현 정확성을 판단할 수 없는 gCAWS는 제외하고 RR·GTO로 원인 분석.

Scheduler insensitivity

기본 환경에서 RR·GTO·gCAWS IPC 차이는 noise 수준 (전구간 0.39% · region 2.15%)

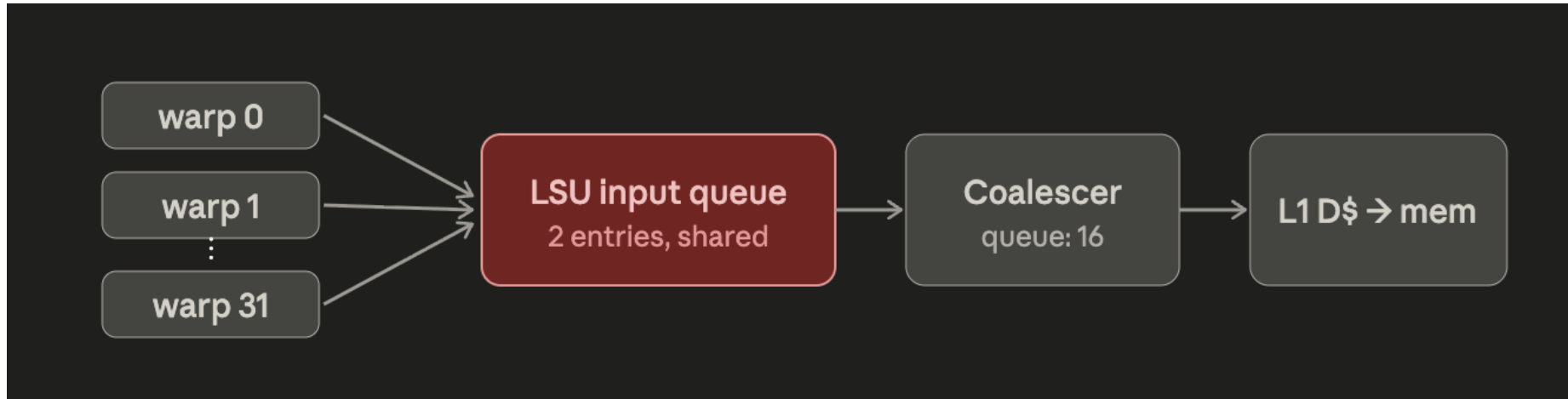
metric	RR	GTO
cycles	584,021	583,217
IPC	6.224	6.233
GTO gain	-	0.14%
L1D misses	37,399	37,074
L1D hit ratio	44%	45%
L1D MPKI	10.288	10.199
same-warp hit rate	1%	1%
inter-warp hit rate	71%	72%
avg streak	1.01	1.01
same wid issue rate	0.52%	0.73%
core LSU pending full	546,699	546,742
coalescer pending full	203,780	201,819

ready warp 수	RR	GTO
0	77%	78%
1	13%	14%
2 – 3	8%	7%
≥ 2 (스케줄 가능)	9%	9%

* 구현 정확성을 판단할 수 없는 gCAWS는 제외하고 RR·GTO로 원인 분석.

lsuq_in = 2인 경우 RR/GTO의 차이가 거의 없을 정도로 LSU stall에 지배된 모습

구조적 원인 — core-wide LSUQ 병목



LSUQ_IN_SIZE = 2 — 코드 분석 결과 32개 warp 전체가 공유하는 core-wide 자원 (NUM_LSU_BLOCKS=1, LsuUnit Core당 1개, 큐 인덱스가 wid가 아닌 block_idx, full 시 전 warp의 load 거절)

코어 전체 동시 load가 2개로 제한(cap=2) → 평균 ready warp $\approx 1.7 \sim 1.8$, **MLP ≈ 1** → 메모리 latency를 숨기지 못함

- coalescer queue(16) · per-warp scoreboard는 병목이 아님

LSUQ 2 → 8 — 1차 병목 정량 확인

지표	LSUQ=2 RR	LSUQ=2 GTO	LSUQ=8 RR	LSUQ=8 GTO
IPC	4.43	4.29	10.45	9.82
scheduler idle	72%	73%	34%	38%
scoreboard stall	72%	73%	34%	38%
L Isu	97%	97%	93%	93%
L fpu	2%	2%	6%	6%
scoreboard_block_ratio	97.36%	97.30%	91.73%	91.77%

IPC ×2.36

RR 4.43 → 10.45

IPC ×2.29

GTO 4.29 → 9.82

idle 72→34% · block_ratio 97.3→91.8%

발견 LSUQ가 1차 병목임을 정량 확인. 단 ready ≥ 2 비율은 9% → 20%로만 늘고 LSUQ를 더 늘려도 saturation → 컴파일러가 노출하는 warp당 MLP 자체가 제한되는 추가 요인 존재.

LSUQ sweep 1차 병목 정량 확인 (MSHR 64) (kmeans f64 p1000, l1d 64kiB 1c32w32t)

LSUQ_IN	policy	cycles	L1 hit	streak avg	max	recent unique32	candidate unique32	ready unique32
2	RR	584,021	44%	1.01	28	8.08	32.00	8.11
2	GTO	583,217	45%	1.01	28	7.99	31.99	8.03
4	RR	459,164	39%	1.00	28	12.99	31.94	13.15
4	GTO	459,163	42%	1.01	22	12.79	31.66	13.04
8	RR	442,040	40%	1.00	13	25.99	31.90	28.93
8	GTO	462,543	40%	1.01	192	20.54	28.55	23.88
16	RR	440,214	40%	1.00	28	26.19	31.85	29.32
16	GTO	461,004	40%	1.01	20	20.32	28.04	24.22
32	RR	438,781	39%	1.00	28	26.10	31.87	29.19
32	GTO	461,073	40%	1.01	28	20.39	27.69	24.60

발견 LSUQ 및 MSHR 확충 시 GTO의 issue pool 제한이 잘 보여짐

blocked same warp issue 원인 점유율

reason	비율
scoreboard total	40.1%
ibuf empty	46.4%
ready waiting	13.0%
none	0.5%

발견 LSUQ 병목 해소한 이후 GTO가 greediness를 보여주지 못한 이유 중 scoreboard stall, ibuf empty가 큰 지분을 가졌음

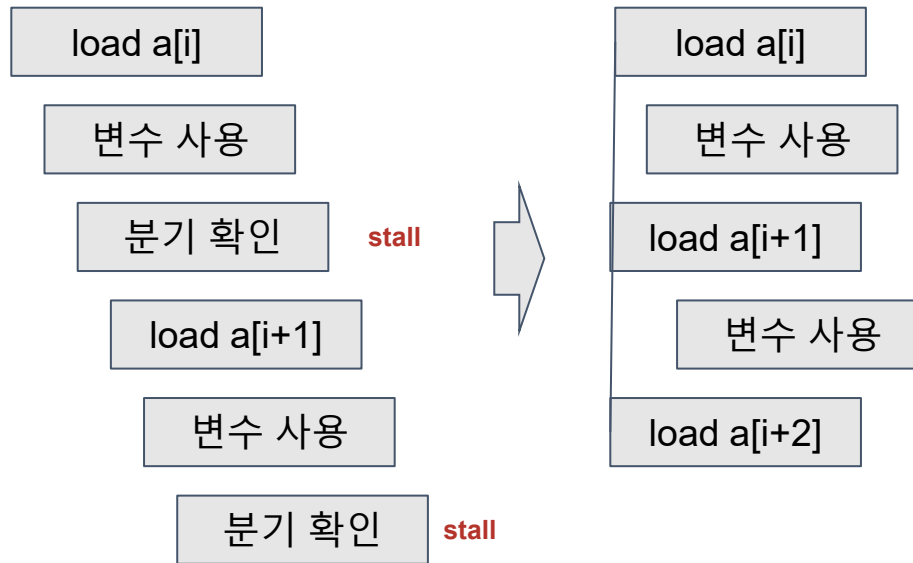
Scoreboard stall 해소: Loop Unrolling을 통한 coalescing 개선

```

$L_BB1_11:
ld.global.f32 %f22, [%rd31+-8];
ld.global.f32 %f23, [%rd32];
sub.f32 %f24, %f23, %f22;
fma.rn.f32 %f25, %f24, %f24, %f53;
add.s64 %rd17, %rd32, %rd4;
ld.global.f32 %f26, [%rd31+-4];
ld.global.f32 %f27, [%rd17];
sub.f32 %f28, %f27, %f26;
fma.rn.f32 %f29, %f28, %f28, %f25;
add.s64 %rd18, %rd17, %rd4;
ld.global.f32 %f30, [%rd31];
ld.global.f32 %f31, [%rd18];
sub.f32 %f32, %f31, %f30;
fma.rn.f32 %f33, %f32, %f32, %f29;
add.s64 %rd19, %rd18, %rd4;
add.s64 %rd32, %rd19, %rd4;
ld.global.f32 %f34, [%rd31+4];
ld.global.f32 %f35, [%rd19];
sub.f32 %f36, %f35, %f34;
fma.rn.f32 %f53, %f36, %f36, %f33;
add.s32 %r72, %r72, 4;
add.s64 %rd31, %rd31, 16;
add.s32 %r71, %r71, -4;
setp.ne.s32 %p12, %r71, 0;
@%p12 bra $L_BB1_11;

```

타 논문에서 사용하는 GPGPU sim에서는
최적화를 통한 4-way unrolling이 됨



unrolling을 통해

- cache coalescing을 통한 MLP 증가
- 루프 블럭을 펼쳐 issue 지속 가능

Vortex에서는 최적화가 명시적으로 필요
필요한 위치에 `#pragma unroll 4` 로 지시

```

16 unsigned int point_id = get_global_id(0);
17 int index = 0;
18 //const unsigned int point_id = get_global_id(0);
19 if (point_id < npoints)
20 {
21     float min_dist=FLT_MAX;
22     for (int i=0; i < nclusters; i++) {
23
24         float dist = 0;
25         float ans = 0;
26 #pragma unroll 4
27         for (int l=0; l<nfeatures; l++){
28             ans += (feature[l * npoints + point_id]-clusters[i*nfeatures+l])*
29                 (feature[l * npoints + point_id]-clusters[i*nfeatures+l]);
30         }
31     }

```


Scoreboard stall 해소: Loop unrolling - 2차 병목 정량 확인 (kmeans f64 p1000, l1d 64kiB 1c32w32t)

LSUQ_IN	policy	cycles	L1 hit	streak avg	max	recent unique32	candidate unique32	ready unique32
2	RR	731,405	40%	1.02	288	8.41	31.84	8.49
2	GTO	729,524	41%	1.03	288	7.96	31.83	8.16
4	RR	596,312	37%	1.01	288	12.65	31.76	12.82
4	GTO	588,872	38%	1.02	288	11.93	31.62	12.3
8	RR	621,962	34%	1.01	288	26.22	31.81	27.87
8	GTO	553,710	36%	1.02	288	15.71	30.2	17.49
16	RR	609,321	35%	1.01	288	26.57	31.77	28.91
16	GTO	549,572	36%	1.02	288	17.42	28.83	20.47
32	RR	617,023	35%	1.01	288	26.38	31.79	28.63
32	GTO	554,600	36%	1.02	288	18.35	27.39	22.79

발견 GTO에서 scoreboard stall을 해소함으로써 IPC 향상 확인 (RR대비 +10%)

Scoreboard stall 해소: ibuffer prefetch - 3차 병목 정량 확인 (kmeans f64 p1000, l1d 64kiB 1c32w32t)

cfg	pending	decode	policy	cycles	L1 hit	MPKI	same hit	inter hit	streak	same-wid	recent32	ibuf empty	SB block	LSUQ full
p1d1	1	1	RR	617,023	35%	15.448	2%	61%	1.01	0.92%	26.38	98%	0%	462,380
p1d1	1	1	GTO	554,600	36%	15.133	11%	59%	1.02	2.26%	18.35	96%	0%	377,676
p2d1	2	1	RR	610,661	35%	15.505	2%	61%	1.02	1.53%	25.84	25%	43%	461,816
p2d1	2	1	GTO	541,471	36%	15.154	9%	59%	1.24	19.64%	16.57	40%	40%	394,241
p4d1	4	1	RR	610,591	35%	15.408	2%	61%	1.02	1.90%	25.6	24%	41%	461,850
p4d1	4	1	GTO	546,014	36%	15.107	11%	58%	1.39	28.20%	16.92	31%	39%	380,858

발견 '비어있어야 ibuffer fill 방식' -> 'ibuffer에 빈공간 있으면 n개의 명령어 미리 fetch하는 방식' 으로 변경하여 ibuffer empty 감소 및 streak 향상. 그러나 L1 hit 및 L1 same warp hit은 여전히 저조함

워크로드 특성 (16 KB L1D)

kmeans

memory · 규칙적 · 균일

- feature 배열 연속 접근 → coalescing 큼
- intra-warp locality 높음
- 모든 warp 동일 루프 횟수 → 진행도 분산 거의 없음

sgemm3

compute · tile locality

- local memory 타일링 + barrier
- tile size 조정 가능
- barrier 근처 latency-hiding warp 부족

bfs

branch · memory · irregular

- id 기반 불규칙 접근 (uncoalesced)
- 데이터 캐시 hit 낮음 (~6~14%)
- 메모리 접근 산발적
- intra-warp locality 활용 가능

워크로드별 실측 결과

스케줄러 우열은 워크로드 구조에 따라 달라짐

워크로드	특성	관측 결과
kmeans	구조적 균일성 · 진행도 분산 없음	input ↑ → RR ≥ GTO (예측과 반대)
sgemm3	tile↑ → barrier 도달 시간↑ → GTO locality 시간↑	tile↑ → GTO > RR (n128/t16 +6.6%)
bfs	uncoalesced · cache hit ~6~14%	RR ≈ GTO (차이 미미)

대표 데이터 sgemm3 n128/t16 — GTO 5.90 vs RR 5.53 IPC (+6.6%) · bfs 4096 nodes — RR 0.871 vs GTO 0.881 IPC

Vortex vs GPGPU-Sim & coalescer 분석

동일 kmeans · L1D 용량 sweep (GPGPU-Sim: 1-SM · 32 thread · 32 warp · No L2 · RR)

Vortex vs GPGPU-Sim

- Vortex GTO는 unique warp 개수를 GPGPU-Sim GTO 수준으로 억제
- 그러나 GTO의 same-warp reuse hit이 예상보다 적음
- 전체 L1D hit은 낮는데 inter-warp reuse가 높게 유지 → GTO가 RR 대비 차이를 못 냄
- inter-warp reuse가 높아 RR의 성능 저하도 거의 없음

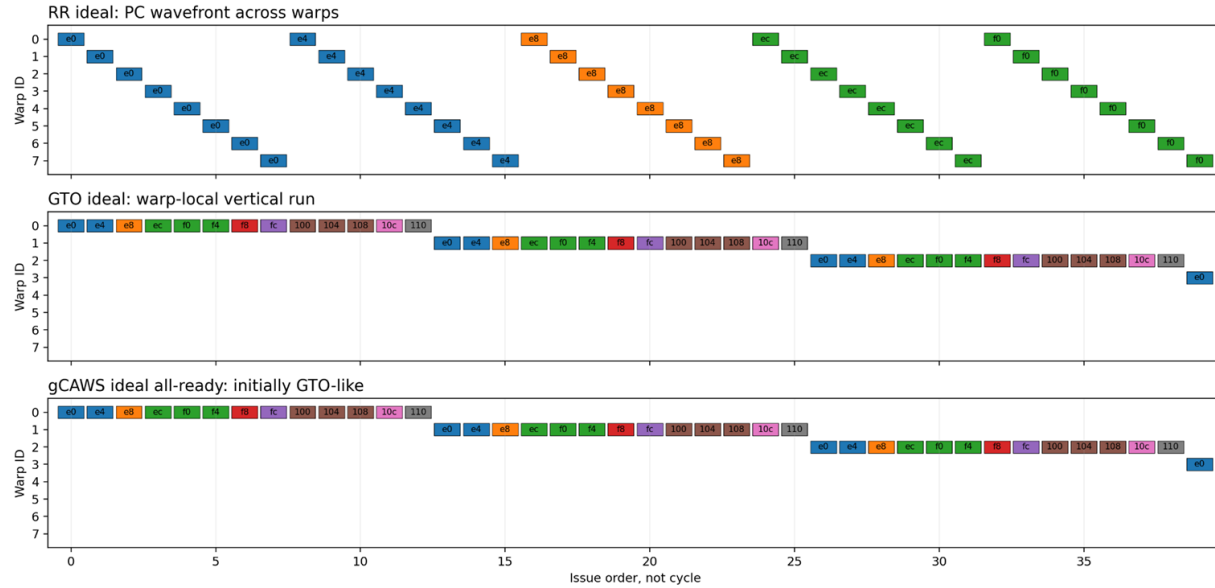
memory coalescer & loop unrolling

같은 cache line인데 여러 request로 분할된 경우 ≈ **3만 건**

- coalescer input당 평균 ~31 load (128B line, 4B×32 → 이상적 1~2 subreq)
- GPGPU-Sim은 load당 < 2 request — 효율 개선 여지
- 4-way loop unrolling → 16 KB부터 GTO same-warp reuse 개선

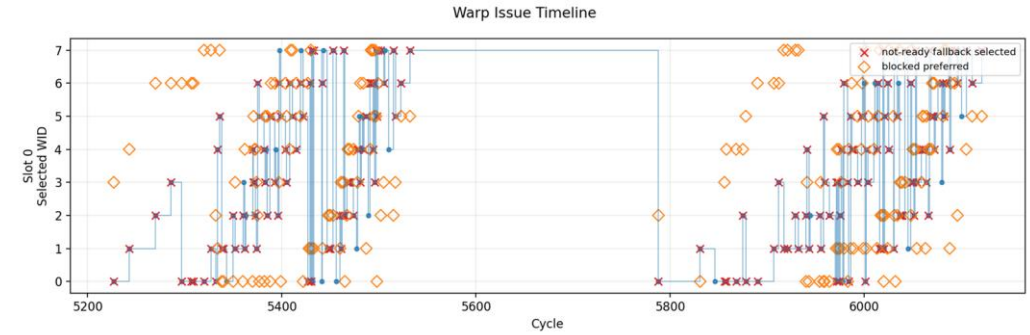
Vortex Arbiter Sanity Check

구현 아이디어 - 동작 간 정합성 확인

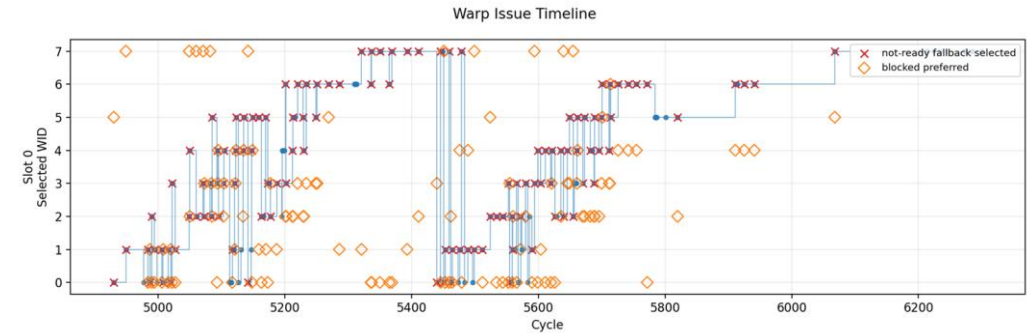


선택된 워프를 그래프로 시각화하여
의도된 동작과 같은지 확인

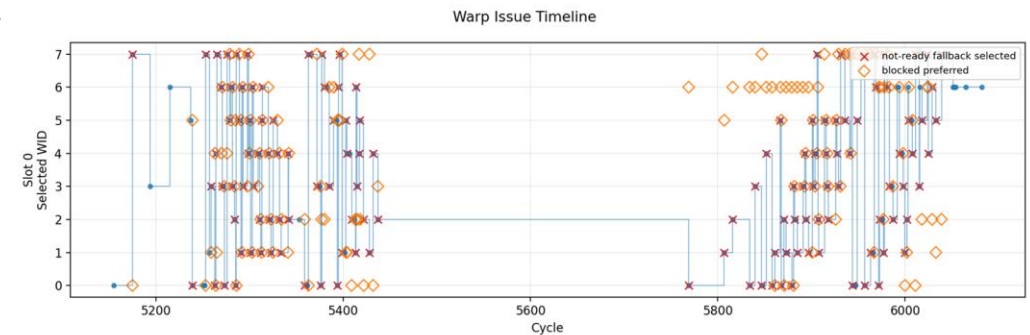
RR



GTO



gCAWS

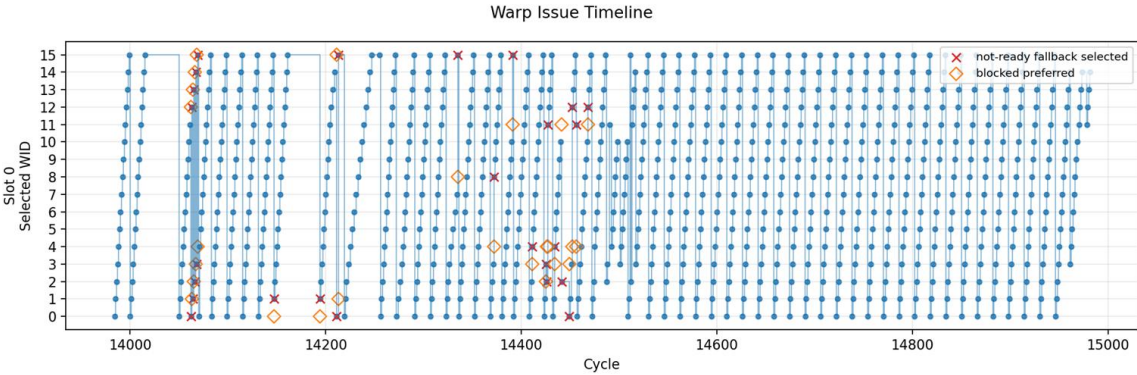


Vortex Arbiter Sanity Check (rtlsim - simx, warps=16, thread=1, arithmetic major benchmark)

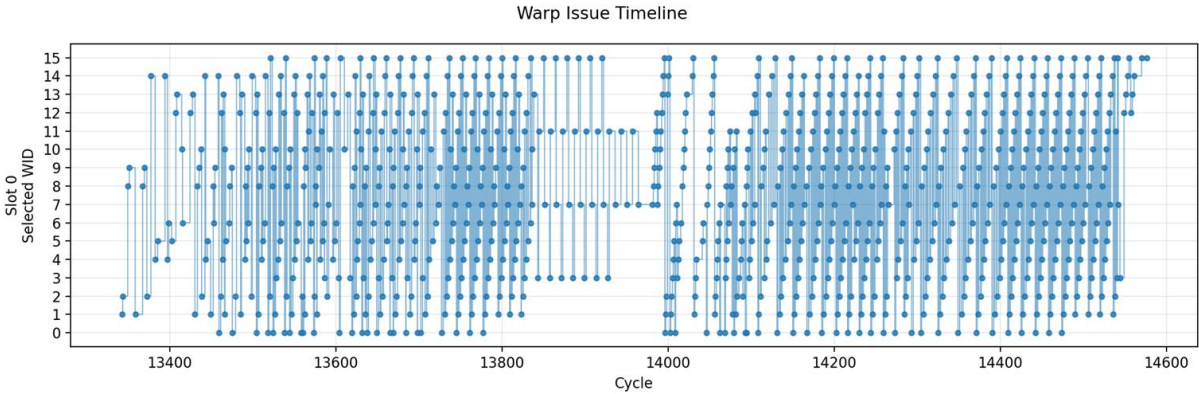
구현 아이디어 - 동작 간 정합성 확인

RR

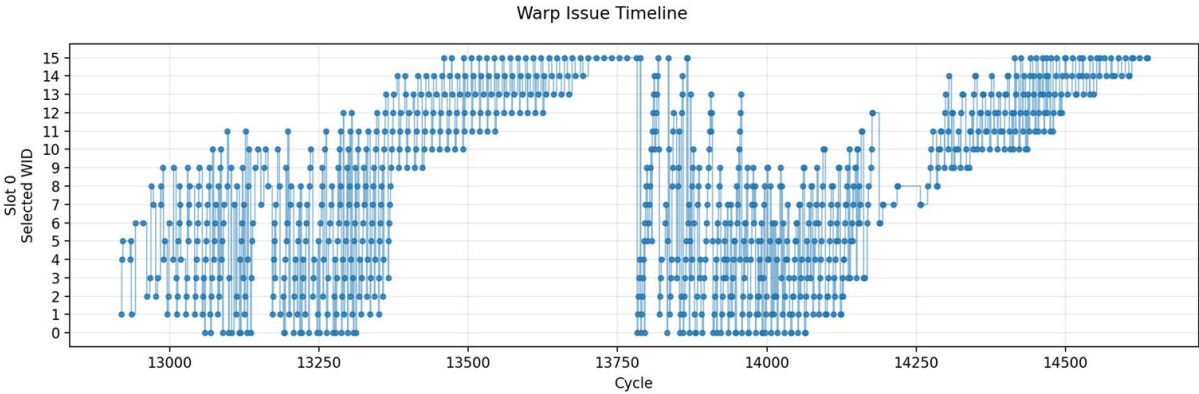
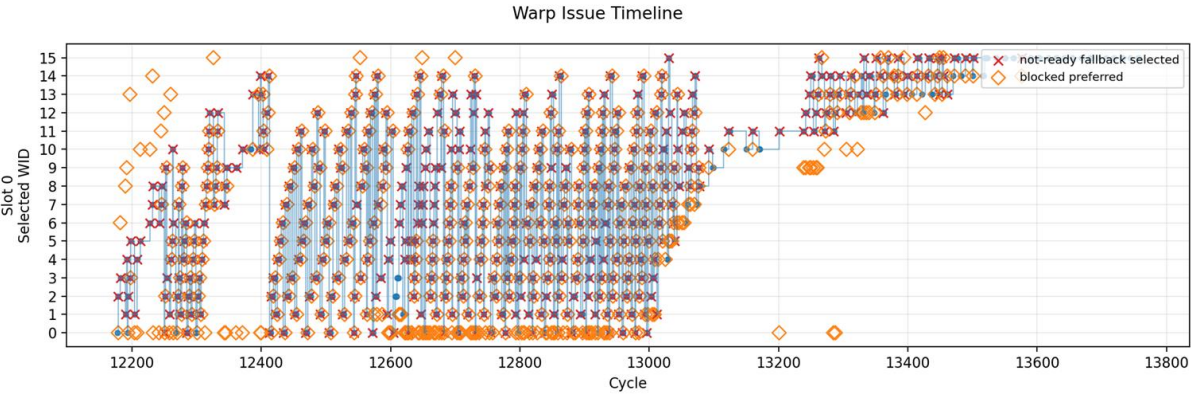
simx



rtlsim



GTO



Vortex Arbiter Sanity Check (rtlsim vcd trace, warps=16, thread=1, arithmetic major benchmark)

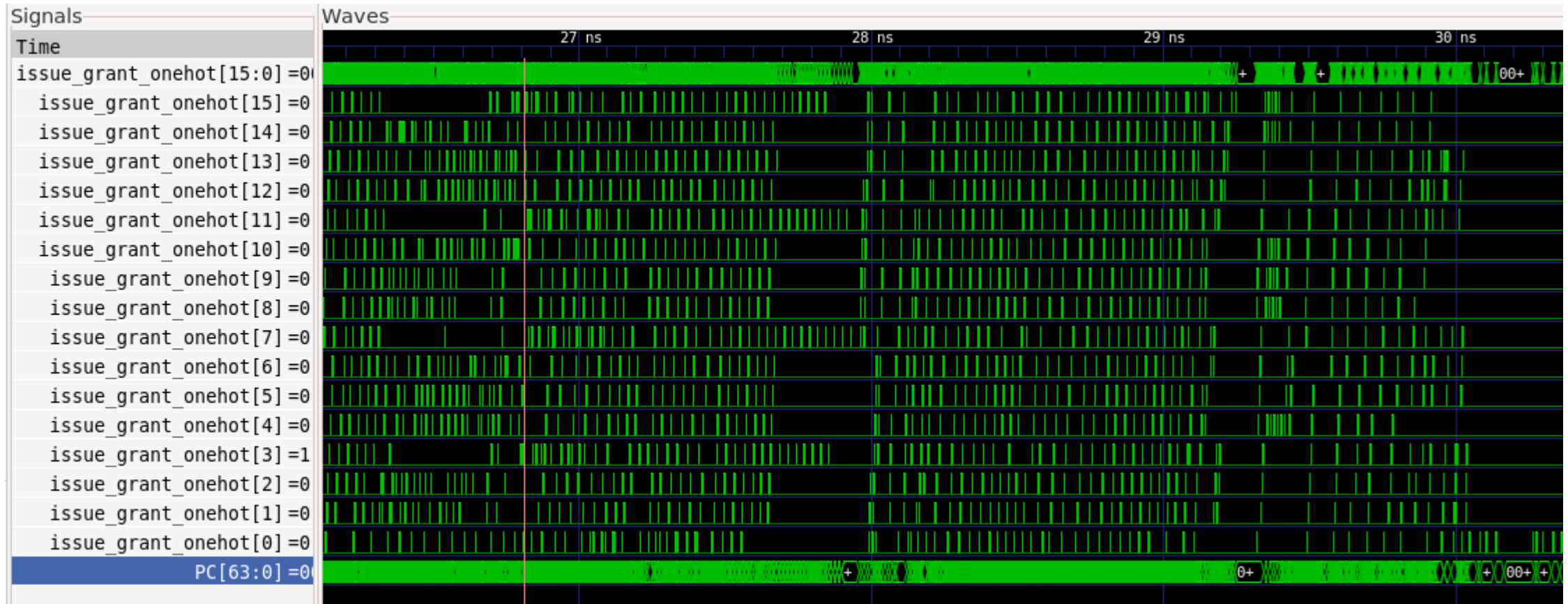
구현 아이디어 - 동작 간 정합성 확인



발견 Verilator의 trace.vcd에서도 비슷한 warp trace가 나옴을 확인함

Vortex Arbiter Sanity Check (rtlsim vcd trace, warps=16, thread=1, arithmetic major benchmark)

구현 아이디어 - 동작 간 정합성 확인



발견 Verilator의 trace.vcd에서도 비슷한 warp trace가 나옴을 확인함

Research Gains

통찰 (INSIGHT)

스케줄러 효과는 조건부

스케줄러 우월은 절대적이지 않고 워프 간 진행도 차이(워크로드 구조 + 메모리 변동)에 좌우된다. 여러 논문들이 전제한 효과가 환경에 따라 사라질 수 있음을 검증.

방법론 (METHOD)

Scheduler insensitivity의 근본 원인 규명

지표(scoreboard_block_ratio · ready 분포) + 체계적 변수 배제 + contention 재현으로 1차 병목(core-wide LSUQ)을 입증.

시뮬레이터 · 마이크로아키텍처를 코드 수준으로 이해

Vortex SimX 내부(LSU 경로 · LSUQ · coalescer · scoreboard · MSHR)와 GPGPU-Sim과의 ISA/toolchain 차이를 체득. SimX → RTL → FPGA 검증 관점 확보.

산출물 (OUTPUT)

재사용 가능한 구현 · 분석 인프라

RR/GTO/gCAWS/iPAWS(fusion) 스케줄러 + 진단·시각화 도구(arbiter 거동 · ready 분포) 구축, Vortex 개선 포인트(LSUQ cap · coalescer 과 분할) 발견.

결론 & 향후 과제

결론

- 1차 원인이 core-wide LSUQ 병목임을 코드 수준에서 규명
- MSHR 축소로 contention 재현 시 gCAWS가 원 논문 효과의 상당 부분(+8~9%) 재현 → contention 의존성 입증
- 추가 제약: 컴파일러의 warp당 MLP 제한 + memory coalescer 비효율

향후 과제 (full-stack)

- ISA 확장을 통한 진정한 의미의 paper-like gCAWS 구현
- memory coalescer 개선 · loop unrolling → warp당 MLP 확대·검증
- 더 큰 입력(L2 초과)에서의 자연적 divergence 유도
- SimX 검증을 RTL 설계 · FPGA로 — 진정한 full-stack